

SeatWise — Phase 3 Summary

Concurrency-Safe Seat Holding — The Core Mechanism

What This Phase Proves

This is the central technical claim of the whole project: when two buyers attempt to hold the same seat at nearly the same moment, the database guarantees exactly one of them succeeds. This phase built that mechanism, wired it into the real seat-selection UI, and — critically — proved it under real concurrent load rather than just reasoning about it.

Design Change From the Original Plan

The original design (specified earlier in this project) used SAVEPOINTS inside a multi-statement transaction. Before writing any code, the actual driver in use — drizzle-orm's neon-http adapter — was checked directly against its source, which throws an explicit error: "No transactions support in neon-http driver." The SAVEPOINT design was not compatible with the driver already chosen for this project.

Replacement design: each seat-hold attempt is a single atomic statement — a data-modifying CTE that deletes any expired hold for that seat, then inserts the new hold with ON CONFLICT (event_id, seat_id) DO NOTHING RETURNING id. If a row comes back, the hold succeeded; if not, someone else already holds that seat. No transaction wrapper, no savepoints, fully compatible with the HTTP-based driver. The buyer's 8-seat cap is enforced separately by a Postgres trigger using pg_advisory_xact_lock, scoped to each single INSERT statement, so two concurrent requests from the same buyer can't both slip under the cap.

```
WITH cleanup AS (  
  DELETE FROM seat_holds  
  WHERE event_id = $1 AND seat_id = $2 AND expires_at < now()  
)  
INSERT INTO seat_holds (id, event_id, seat_id, buyer_id, expires_at)  
VALUES ($3, $1, $2, $4, $5)  
ON CONFLICT (event_id, seat_id) DO NOTHING  
RETURNING id
```

What Was Built

File	Purpose
Postgres trigger (Neon)	enforce_buyer_hold_cap() + seat_holds_enforce_cap trigger — rejects a hold attempt that w
src/lib/db/holds.ts	Core logic: holdSeatsCore, releaseSeatCore, getHeldOrSoldSeats. Testable directly, indepe
src/lib/db/seats.ts (Server Actions)	holdSeats, releaseSeat, getSeatStatus — pulls the real buyer id from the authenticated sess
src/app/components/TicketDrawer.tsx	Static bookedSeats array (12 hardcoded fake seats) removed. Seat availability now fetched
src/app/events/[id]/page.tsx	event.id now passed into the drawer — previously missing, which caused every hold attempt
src/lib/db/concurrency-test.ts	Automated proof: creates a throwaway event and 20 throwaway buyers, fires 20 parallel hol

Problems Faced and Fixed

- 1. Missing event id in the drawer.** The TicketDrawer prop object never included event.id, so every hold/release/status call silently received undefined. Postgres correctly rejected the malformed query, but the browser's error overlay initially showed a sanitized, misleading version of the error with empty parameters — the real cause was only visible in the terminal running npm run dev, which showed getSeatStatus(undefined) and holdSeats(undefined, [...]) directly.
- 2. A stray duplicate JSX tag** was introduced while patching in the id field, producing confusing TypeScript errors ("expected 0 type arguments") that looked unrelated to the actual typo. Root cause was two stacked opening <TicketDrawer tags instead of one.
- 3. The buyer-cap error was never actually being caught.** The automated test caught a real bug in production code, not just in the test itself: the Postgres error code (P0001) lives on err.cause.code, not err.code directly, because drizzle wraps the underlying Neon error inside its own DrizzleQueryError. The original catch block checked err.code, which never matched, so a buyer hitting the 8-seat cap would have crashed with a generic error instead of the intended "you've reached the seat limit" message. Fixed by checking err.cause?.code ?? err.code.

Verification Performed

Two separate levels of proof, deliberately not just one:

Test	Result
Local Postgres, raw TCP connections (pg driver), 30 runs x 20 parallel requests, 10 concurrent, every run	requests, 10 concurrent, every run
Local Postgres, buyer cap, 10 parallel requests from one buyer	8 held / 2 rejected
Real Neon database (via app's actual neon-http driver), 10 runs x 20 parallel requests, 10 concurrent, every run	requests, 10 concurrent, every run
Real Neon database, buyer cap, 10 parallel requests from one buyer	8 held / 2 cap_exceeded
Manual two-browser test (two different logged-in accounts, same seat, same event)	One session held the seat; the other received a real conflict message

The local Postgres test (via true simultaneous TCP connections) proves the SQL pattern itself is race-free. The Neon test proves the same guarantee holds through the actual driver and network path this app uses in production. Testing both, rather than just one, was deliberate — passing only the local test would not have proven the real app's wiring was correct, and passing only the Neon test would not have proven the underlying mechanism is genuinely race-free rather than coincidentally correct under light load.

Known Gap Carried Forward (Not a Bug — Expected, Documented)

A hold is temporary by design (10-minute expiry) and is never converted into a permanent sale, because real payment processing does not exist yet. Observed directly: after "confirming" a booking through the current mock payment flow, the seat is only held, not sold — once the hold expires, or once a second browser session re-fetches seat status, the seat correctly becomes available again in the database. This is the accurate, correct behavior of an unconverted hold, not a defect in the concurrency logic. Converting a paid hold into a permanent ticket is Phase 5.

Separately, seat status in a given browser session is only fetched once, when the drawer opens — there is no live update if another buyer holds a seat while your drawer is already open. This is the explicitly scoped gap that Phase 4 (SSE) exists to close.